

NF06_EN - Practice of programming – P26
Project – E-commerce Delivery Management System in C

1. Project Description

The objective is to develop an **e-commerce delivery management system** using the C programming language.

The system allows customers to browse products, place orders, and simulate deliveries. An administrator can manage deliveries and track sales.

Orders will be managed using **FIFO queues implemented with linked lists (First In, First Out)**, representing deliveries to be completed.

Students will work in **pairs or groups of three** to design and test the system.

Stock Management

At program startup, the product stock is initialized from a **text file or a CSV file**.

During execution:

- Stock decreases when an order is validated
- No restocking is performed in this version

2. Features and Requirements

2.1. Product Data

- Product information is loaded from the product stock file
- Each product contains:
 - Category (e.g., Electronics, Clothing, Food, etc.)
 - Product name
 - Price
 - Stock

Stock rules

- -1 → unlimited stock
- 0 → unavailable
- >0 → available quantity

Example CSV format

Electronics

Laptop_1, 1200, 5

Laptop_2, 600, 10

Smartphone_1, 200, 20

Smartphone_2, 800, 10

Headphones_1, 250, 15

Headphones_2, 30, -1

Clothing

Tshirt_1, 20, 30

Jeans_1, 50, 15

Food

Chocolate, 3, 100

Coffee, 5, -1

Each group must create its own file with at least **two additional categories and products**.

2.2. Data Handling and Stock Management

Product data must be loaded into memory at program startup.

Requirements

- Product data must be stored in a **table (array) of structures** of type `Product`
- The file must be read **only once at the beginning of the program**
- During execution, all operations must be performed using the **in-memory data (the array)**

Stock Update

- Stock values must be updated **in the array** after each validated order
- The file must **not be accessed again during execution**

End of Program Behavior

When the user chooses to exit the program:

- The updated product data must be written back to the file
- The file must reflect the **final state of the stock**

Constraints

- The use of the structure `Product` is **mandatory**
- The use of parallel arrays is not allowed
- All stock operations must be performed on the **in-memory data structure**

Suggested Structure

```
typedef struct {
    char name[50];
    char category[30];
    float price;
    int stock;
} Product;
```

2.3. Customer Profile

The system allows a customer to:

- View product categories
- Browse products
- Add products to a cart (multiple items allowed)
- View the total price
- Choose a priority level for the order
- Enter a delivery address
- Validate the order

Stock rules (customer side)

- It is not possible to add more than the available stock (except for -1)
- Products with stock = 0 are unavailable
- Stock is updated **only when the order is validated**

After validation

The order is added to one of the **two delivery queues (FIFO)** depending on its priority:

- Priority 1 queue (high priority)
- Priority 2 queue (normal priority)

2.4. Administrator Profile

The administrator has access to:

- Process deliveries according to the **priority scheduling rule**
- View statistics at any time:
 - Total number of deliveries completed
 - Total revenue generated

Statistics must be updated after each delivery.

2.5. Queue and Data Structures

Orders must be managed using **two separate queues**, each implemented as a **FIFO linked list**:

- One queue for **priority 1 orders**
- One queue for **priority 2 orders**

Suggested structure

```
typedef struct Order {  
    int id;  
    char address[100];  
    float total;  
    int priority; // 1 = high, 2 = normal  
    struct Order* next;  
} Order;
```

All suggested data structures can be extended if needed.

Order Identification

- Order IDs must be **generated automatically**
- They must be **unique and strictly increasing**

Required operations

For each queue:

- enqueue() → add an order to the end of the queue
- dequeue() → remove the first order
- isEmpty() → check if the queue is empty

Processing Rule

When the administrator processes deliveries:

- Process **two orders from priority 1**, then
- Process **one order from priority 2**

Special cases:

- If the priority 1 queue is empty → process priority 2
- If the priority 2 queue is empty → process priority 1

2.6. Delivery History (File Management)

The system must keep a history of delivered orders in a text file.

File Name

- The file name must correspond to today's date
- Format: DD-MM-YYYY.txt
Example: 26-03-2026.txt

File Content

Each delivered order must be appended to the daily history file.

Each line contains:

- Order ID
- Delivery address
- Total price

2.7. Command Line Interface (CLI)

The system must run in a **command-line interface**.

The program should display a menu allowing the user to select:

1. Customer mode
2. Administrator mode
3. Exit

It must then:

- Guide the user through corresponding menus
- Handle invalid inputs
- Allow clean program exit

3. Deliverables

Each group must submit on Moodle:

- The complete C project
- The text or CSV files
- A short report (8–10 pages) explaining:
 - Data structures
 - Main functions
 - Design choices
- Presentation slides explaining how the system works

4. Important Dates

- **Submission deadline:** Monday, June 15 at 6 PM
- **Oral presentations:**
 - Tuesday, June 16 (lecture session)
 - Wednesday, June 17 (lab session)
 - Friday, June 19 (lab session)
- The schedule will be published on Moodle.

Each group will have:

- 5 minutes presentation
- 5 minutes questions